

```

"""
config.py
=====

FUENTE UNICA DE VERDAD de la configuracion del proyecto.

Los nombres de las columnas de rol (`columna_target`, `columna_id`,
`columna_tiempo`) viven UNICAMENTE aqui / en `config.yaml`. Ningun otro modulo
del proyecto escribe literales como "target" o "periodo": todos reciben el
objeto :class:`ConfigPipeline` y leen `cfg.columna_target`, `cfg.columna_id`,
`cfg.columna_tiempo`. Cambiar el nombre en `config.yaml` (o por CLI) es
suficiente para que TODO el pipeline –incluido el generador de datos de
demostracion y el reporte Excel– se adapte solo.

Precedencia de configuracion (de menor a mayor prioridad):

    defaults del dataclass < config.yaml < argumentos de linea de comandos
"""

from __future__ import annotations

import json
from dataclasses import asdict, dataclass, field, fields
from pathlib import Path
from typing import Any

from .logging_utils import obtener_logger

LOGGER = obtener_logger("config")

# -----
# Excepcion de dominio
# -----
class ErrorConfiguracion(ValueError):
    """Se lanza cuando la configuracion es invalida o inconsistente."""

# -----
# Configuracion
# -----
@dataclass
class ConfigPipeline:
    """Parametros completos del pipeline de seleccion de variables.

    Se divide en bloques: entradas obligatorias, umbrales por fase y opciones
    de ejecucion. Todos los umbrales son parametrizables; los valores por
    defecto son los justificados en la documentacion (`docs/documentacion.html`).
    """

    # =====
    # BLOQUE A. Entradas obligatorias del proceso
    # =====
    ruta_dataset: str = "data/panel_sintetico.csv"
    columna_target: str = "target"
    columna_id: str = "id_entidad"
    columna_tiempo: str = "periodo"
    usar_boruta: bool = True
    ruta_salida_excel: str = "outputs/bitacora_feature_selection.xlsx"

    #: Columnas adicionales que deben excluirse de la evaluacion sin analizarlas
    #: (identificadores secundarios, campos de auditoria, llaves foraneas...).
    columnas_excluidas: list[str] = field(default_factory=list)

    #: Si True, ademas del Excel de bitacora se exporta un dataset "listo para
    #: modelar": id + tiempo + target + solo las variables que superaron las
    #: tres fases obligatorias (univariada, bivariada y multivariada).
    exportar_dataset_final: bool = True
    #: Ruta del dataset final. Si se deja vacio, se deriva automaticamente junto
    #: a `ruta_salida_excel` como "<mismo_nombre>_dataset_final.<formato>".
    ruta_dataset_final: str = ""

```

```

#: Formato de escritura del dataset final: "csv" | "parquet".
formato_dataset_final: str = "csv"

# =====
# BLOQUE B. Fase 1 - Univariado
# =====
#: Umbral PRINCIPAL de (%ceros + %nulos). >= a esto -> se elimina.
#: 0.95 = la variable es constante en el 95% de las filas: aporta señal
#: solo en <=5% de la muestra y en panel eso suele ser ruido de un puñado
#: de entidades.
umbral_ceros_nulos: float = 0.95
#: Umbral ALTERN0, mas conservador. Se activa con `usar_umbral_alterno`.
umbral_ceros_nulos_alterno: float = 0.90
usar_umbral_alterno: bool = False

#: Desviacion estandar por debajo de la cual se considera constante.
umbral_std_minimo: float = 1e-8
#: Coeficiente de variacion (std/|media|) minimo. Mide dispersion RELATIVA:
#: una std de 0.01 es despreciable si la media es 1e6, pero enorme si la
#: media es 0.001. Por eso no basta con mirar la std absoluta.
umbral_cv_minimo: float = 1e-4
#: Proporcion maxima admisible del valor/categoria mas frecuente.
#: >= 0.99 -> una sola categoria domina y la variable es casi constante.
umbral_dominancia: float = 0.99
#: IQR (p75-p25) por debajo del cual se marca "percentiles comprimidos".
umbral_iqr_minimo: float = 0.0
#: Numero minimo de valores unicos para considerar evaluable una variable.
minimo_valores_unicos: int = 2

# =====
# BLOQUE C. Fase 2 - Bivariado (IV / Gini)
# =====
#: Numero de bins objetivo para la discretizacion por cuantiles (WOE).
n_bins: int = 10
#: Fraccion minima de la muestra por bin. Bins mas chicos se fusionan:
#: un bin con 5 observaciones produce un WOE inestable y un IV inflado.
min_prop_bin: float = 0.03
#: Cardinalidad maxima de una categorica; el resto se agrupa en "OTROS".
max_categorias: int = 50
#: Correccion de continuidad (Haldane-Anscombe) para evitar log(0) en WOE.
correccion_woe: float = 0.5

#: Pesos del score compuesto. Por defecto balanceado 50/50.
peso_gini: float = 0.50
peso_iv: float = 0.50
#: Normalizacion previa a la ponderacion: "minmax" | "rank".
metodo_normalizacion: str = "minmax"

#: Pisos de poder predictivo. Regla de exclusion: se descarta la variable
#: solo si falla en AMBAS metricas (IV bajo Y Gini bajo), para no penalizar
#: variables que una de las dos metricas capta mejor.
umbral_iv_minimo: float = 0.02
umbral_gini_minimo: float = 0.05
#: Piso opcional sobre el score compuesto normalizado (0 = desactivado).
umbral_score_minimo: float = 0.0

#: Piso de ruido estadistico. Los umbrales fijos (0.02 / 0.05) no dependen
#: del tamano de la muestra ni del numero de bins, pero el IV espurio de una
#: variable aleatoria SI depende de ambos. Con esto activado, el umbral
#: efectivo es el MAYOR entre el fijo y el piso de ruido calculado.
usar_piso_ruido: bool = True
#: Nivel de significancia del contraste contra la hipotesis de irrelevancia.
alpha_ruido: float = 0.01
#: Corregir alpha por el numero de variables evaluadas (Bonferroni).
#: Mas estricto: reduce falsos positivos a costa de perder senales debiles.
bonferroni_ruido: bool = False
#: IV por encima del cual se sospecha fuga de informacion (leakage).
umbral_iv_sospechoso: float = 0.50
#: Si True, las variables con sospecha de fuga se excluyen ademas de marcarse.
excluir_sospecha_fuga: bool = False
#: Si >0, conserva solo las N mejores por score compuesto.

```

```

top_n_bivariado: int = 0

#: Estabilidad temporal (especifico de panel): PSI maximo tolerado entre
#: el periodo base y el resto. >0.25 = cambio poblacional severo.
umbral_psi: float = 0.25
#: Si True, la inestabilidad temporal tambien excluye (por defecto solo marca).
excluir_por_inestabilidad: bool = False

# =====
# BLOQUE D. Fase 3 - Multivariado
# =====
#: Umbral de asociacion absoluta para declarar redundancia.
umbral_correlacion: float = 0.90
#: Metodo para pares numerico-numerico: "spearman" | "pearson".
metodo_correlacion: str = "spearman"
#: VIF por encima del cual se marca multicolinealidad severa.
umbral_vif: float = 10.0
#: Si True, ademas de marcar, el VIF elimina iterativamente.
excluir_por_vif: bool = False

# =====
# BLOQUE E. Fase 4 - Boruta (opcional)
# =====
#: "auto" (libreria si existe, si no nativo) | "borutapy" | "borutashap" | "nativo"
motor_boruta: str = "auto"
boruta_n_estimadores: int = 200
boruta_max_iter: int = 60
boruta_alpha: float = 0.05
boruta_profundidad_max: int = 6
#: Submuestreo para acotar el costo de Boruta en paneles grandes (0 = sin limite).
boruta_max_filas: int = 100_000

# =====
# BLOQUE F. Ejecucion
# =====
semilla: int = 42
n_jobs: int = -1
autoinstalar_dependencias: bool = True
ruta_log: str = "outputs/featsel.log"
nivel_log: str = "INFO"
#: Separador y encoding para datasets CSV.
csv_sep: str = ","
csv_encoding: str = "utf-8"
#: Si el dataset no existe, generar el panel sintetico de demostracion.
generar_demo_si_falta: bool = True

# -----
# Propiedades derivadas
# -----
@property
def columnas_rol(self) -> list[str]:
    """Columnas reservadas (target, id, tiempo). Nunca son candidatas."""
    return [self.columna_target, self.columna_id, self.columna_tiempo]

@property
def columnas_no_candidatas(self) -> list[str]:
    """Roles reservados + exclusiones explicitas del usuario."""
    return self.columnas_rol + list(self.columnas_excluidas)

@property
def umbral_ceros_nulos_efectivo(self) -> float:
    """Umbral realmente aplicado en la fase 1.1 (principal o alternativo)."""
    return self.umbral_ceros_nulos_alterno if self.usar_umbral_alterno else self.umbral_ceros_nulos

@property
def ruta_dataset_final_efectiva(self) -> Path:
    """Ruta real del dataset final, derivandola si el usuario no la fija.

    Se deriva junto al Excel de bitacora (mismo directorio, mismo nombre
    base) para que ambos artefactos de una misma corrida queden agrupados
    y sea evidente a que bitacora corresponde cada dataset exportado.

```

```

"""
if self.ruta_dataset_final.strip():
    return Path(self.ruta_dataset_final)
base = Path(self.ruta_salida_excel)
extension = ".parquet" if self.formato_dataset_final == "parquet" else ".csv"
return base.with_name(f"{base.stem}_dataset_final{extension}")

def rol_de(self, columna: str) -> str:
    """Clasifica una columna segun su papel en el panel."""
    if columna == self.columna_target:
        return "TARGET"
    if columna == self.columna_id:
        return "ID_ENTIDAD"
    if columna == self.columna_tiempo:
        return "TIEMPO"
    if columna in self.columnas_excluidas:
        return "EXCLUIDA_MANUAL"
    return "CANDIDATA"

# -----
# Serializacion
# -----
def a_dict(self) -> dict[str, Any]:
    """Diccionario plano (para la hoja de parametros del Excel)."""
    return asdict(self)

def a_filas(self) -> list[dict[str, Any]]:
    """Filas parametro/valor/bloque para la bitacora."""
    bloques = {
        "ruta_dataset": "A. Entradas", "columna_target": "A. Entradas",
        "columna_id": "A. Entradas", "columna_tiempo": "A. Entradas",
        "usar_boruta": "A. Entradas", "ruta_salida_excel": "A. Entradas",
        "columnas_excluidas": "A. Entradas",
        "exportar_dataset_final": "A. Entradas", "ruta_dataset_final": "A. Entradas",
        "formato_dataset_final": "A. Entradas",
    }
    filas = []
    for f in fields(self):
        valor = getattr(self, f.name)
        if isinstance(valor, (list, dict)):
            valor = json.dumps(valor, ensure_ascii=False)
        filas.append(
            {
                "parametro": f.name,
                "valor": valor,
                "bloque": bloques.get(f.name, _bloque_por_prefijo(f.name)),
                "tipo": type(getattr(self, f.name)).__name__,
            }
        )
    return filas

# -----
# Validacion
# -----
def validar(self) -> None:
    """Valida coherencia interna. Se ejecuta ANTES de tocar el dataset.

    Raises
    -----
    ErrorConfiguracion
        Ante cualquier parametro fuera de rango o combinacion imposible.
    """
    errores: list[str] = []

    # --- Entradas obligatorias no vacias -----
    for nombre in ("ruta_dataset", "columna_target", "columna_id",
                  "columna_tiempo", "ruta_salida_excel"):
        valor = getattr(self, nombre)
        if not isinstance(valor, str) or not valor.strip():
            errores.append(f"'{nombre}' es obligatorio y debe ser un texto no vacio.")

```

```

# --- Las tres columnas de rol deben ser distintas entre si -----
roles = [self.columna_target, self.columna_id, self.columna_tiempo]
if len(set(roles)) != 3:
    errores.append(
        f"columna_target/columna_id/columna_tiempo deben ser distintas entre si; "
        f"se recibio {roles}."
    )
# --- ...y no pueden estar en la lista de exclusion manual -----
choque = set(roles) & set(self.columnas_excluidas)
if choque:
    errores.append(f"Columnas de rol listadas tambien en columnas_excluidas: {sorted(choque)}.")

# --- Rangos de proporciones -----
for nombre in ("umbral_ceros_nulos", "umbral_ceros_nulos_alterno",
               "umbral_dominancia", "umbral_correlacion", "min_prop_bin"):
    valor = getattr(self, nombre)
    if not 0.0 < float(valor) <= 1.0:
        errores.append(f"'{nombre}'={valor} debe estar en el intervalo (0, 1].")

if self.umbral_ceros_nulos_alterno > self.umbral_ceros_nulos:
    errores.append(
        "El umbral alterno debe ser MAS conservador (menor) que el principal: "
        f"{self.umbral_ceros_nulos_alterno} > {self.umbral_ceros_nulos}."
    )

# --- Pesos del score compuesto -----
suma = self.peso_gini + self.peso_iv
if suma <= 0:
    errores.append("peso_gini + peso_iv debe ser mayor que cero.")
elif abs(suma - 1.0) > 1e-9:
    LOGGER.warning(
        "peso_gini + peso_iv = %.4f != 1. Se renormalizaran a %.3f / %.3f.",
        suma, self.peso_gini / suma, self.peso_iv / suma,
    )

if self.metodo_normalizacion not in ("minmax", "rank"):
    errores.append(f"metodo_normalizacion='{self.metodo_normalizacion}' no valido (minmax|rank).")
if self.metodo_correlacion not in ("spearman", "pearson"):
    errores.append(f"metodo_correlacion='{self.metodo_correlacion}' no valido (spearman|pearson).")
if self.motor_boruta not in ("auto", "borutapy", "borutashap", "nativo"):
    errores.append(f"motor_boruta='{self.motor_boruta}' no valido (auto|borutapy|borutashap|nativo).")
if self.formato_dataset_final not in ("csv", "parquet"):
    errores.append(f"formato_dataset_final='{self.formato_dataset_final}' no valido (csv|parquet).")

# --- Enteros positivos -----
if self.n_bins < 2:
    errores.append(f"n_bins={self.n_bins} debe ser >= 2.")
if self.max_categorias < 2:
    errores.append(f"max_categorias={self.max_categorias} debe ser >= 2.")
if self.boruta_max_iter < 5:
    errores.append(f"boruta_max_iter={self.boruta_max_iter} debe ser >= 5.")
if not 0 < self.boruta_alpha < 1:
    errores.append(f"boruta_alpha={self.boruta_alpha} debe estar en (0, 1).")

if errores:
    raise ErrorConfiguracion(
        "Configuracion invalida:\n - " + "\n - ".join(errores)
    )

# Normalizacion de pesos (post-validacion, para que sumen 1 exacto).
total = self.peso_gini + self.peso_iv
self.peso_gini /= total
self.peso_iv /= total

LOGGER.info("Configuracion validada correctamente.")
LOGGER.info(
    "Columnas de rol -> target='%s' | id='%s' | tiempo='%s'",
    self.columna_target, self.columna_id, self.columna_tiempo,
)

```

```

def _bloque_por_prefijo(nombre: str) -> str:
    """Asigna un bloque legible a cada parametro para la hoja de bitacora."""
    if nombre.startswith(("umbral_ceros", "umbral_std", "umbral_cv", "umbral_dominancia",
                          "umbral_iqr", "minimo_valores", "usar_umbral")):
        return "B. Univariado"
    if nombre.startswith(("n_bins", "min_prop", "max_categorias", "correccion_woe",
                          "peso_", "metodo_normalizacion", "umbral_iv", "umbral_gini",
                          "umbral_score", "excluir_sospecha", "top_n", "umbral_psi",
                          "excluir_por_inestabilidad", "usar_piso_ruido",
                          "alpha_ruido", "bonferroni_ruido")):
        return "C. Bivariado"
    if nombre.startswith(("umbral_correlacion", "metodo_correlacion", "umbral_vif", "excluir_por_vif")):
        return "D. Multivariado"
    if nombre.startswith(("motor_boruta", "boruta_")):
        return "E. Boruta"
    return "F. Ejecucion"

# -----
# Carga desde YAML / dict
# -----
def cargar_config(
    ruta_yaml: str | Path | None = None,
    overrides: dict[str, Any] | None = None,
) -> ConfigPipeline:
    """Construye la configuracion aplicando la precedencia documentada.

    Parameters
    -----
    ruta_yaml
        Ruta al `config.yaml`. Si no existe o PyYAML no esta disponible, se
        usan los defaults del dataclass y se registra la situacion.
    overrides
        Valores de mayor prioridad (tipicamente los flags del CLI). Las claves
        con valor ``None`` se ignoran para no pisar el YAML con "no informado".
    """
    datos: dict[str, Any] = {}

    if ruta_yaml is not None:
        ruta_yaml = Path(ruta_yaml)
        if ruta_yaml.is_file():
            try:
                import yaml # import diferido: puede haberse instalado en bootstrap

                with ruta_yaml.open("r", encoding="utf-8") as fh:
                    crudo = yaml.safe_load(fh) or {}
                    datos = _aplanar_yaml(crudo)
                    LOGGER.info("Configuracion leida desde %s (%d parametros).", ruta_yaml, len(datos))
            except ImportError:
                LOGGER.warning("PyYAML no disponible; se usan los valores por defecto del codigo.")
            except Exception as exc: # noqa: BLE001
                raise ErrorConfiguracion(f"No se pudo leer '{ruta_yaml}': {exc}") from exc
        else:
            LOGGER.warning("No se encontro '%s'; se usan los valores por defecto.", ruta_yaml)

    if overrides:
        limpios = {k: v for k, v in overrides.items() if v is not None}
        if limpios:
            LOGGER.info("Overrides de linea de comandos: %s", limpios)
        datos.update(limpios)

    validos = {f.name for f in fields(ConfigPipeline)}
    desconocidos = set(datos) - validos
    if desconocidos:
        LOGGER.warning("Parametros desconocidos ignorados: %s", sorted(desconocidos))

    cfg = ConfigPipeline(**{k: v for k, v in datos.items() if k in validos})
    cfg.validar()
    return cfg

```

```
def _aplanar_yaml(crudo: dict[str, Any]) -> dict[str, Any]:
    """Aplana un YAML organizado en secciones a un dict de un solo nivel.

    Permite escribir `config.yaml` agrupado por bloques (mas legible) sin que
    el dataclass tenga que conocer esa estructura.
    """
    plano: dict[str, Any] = {}
    for clave, valor in crudo.items():
        if isinstance(valor, dict):
            plano.update(_aplanar_yaml(valor))
        else:
            plano[clave] = valor
    return plano
```